

ONBOARDING & CONTRIBUTOR GUIDE

# AI Agent Governance Platform

Everything a new developer needs to get the project running locally and understand where things live.

## 1. Prerequisites

- Docker + Docker Compose (recommended path — brings up every service with one command)
- Node.js 20+ and npm (if running the frontend outside Docker)
- Go 1.22+ (if running the Gateway outside Docker)
- Java 21 + Maven (if running the Control Plane outside Docker)
- Python 3.12+ (if running the Agent Runner outside Docker)

## 2. Quick Start (Docker Compose)

```
git clone <repo-url>
cd ai-agent-governance-platform
cp .env.example .env
docker compose up --build
```

This brings up Postgres (with the schema in db/migrations applied automatically), Redis, OPA, the Gateway, the Control Plane, the Agent Runner, and the frontend.

## 3. Service Ports

| Service                     | Port | URL                   |
|-----------------------------|------|-----------------------|
| Frontend (Next.js)          | 3000 | http://localhost:3000 |
| Control Plane (Spring Boot) | 8081 | http://localhost:8081 |
| Gateway (Go)                | 8080 | http://localhost:8080 |
| Agent Runner (Python)       | 8000 | http://localhost:8000 |
| Postgres                    | 5432 | localhost:5432        |
| Redis                       | 6379 | localhost:6379        |
| OPA                         | 8181 | http://localhost:8181 |

## 4. Folder-by-Folder Guide

| Folder            | What lives here                                                    |
|-------------------|--------------------------------------------------------------------|
| frontend/         | Next.js dashboard — agent registry, approval queue, audit explorer |
| services/gateway/ | Go — the Agent Gateway (permission + policy enforcement, hot path) |

| Folder                  | What lives here                                                          |
|-------------------------|--------------------------------------------------------------------------|
| services/control-plane/ | Spring Boot — identity, permissions admin, policy management, approvals  |
| services/agent-runner/  | Python/FastAPI — mock agents for local demo and integration testing      |
| policies/               | OPA/Rego policy definitions                                              |
| db/migrations/          | SQL schema migrations, run automatically by docker-compose on first boot |
| infra/                  | docker-compose (local) and, later, Kubernetes manifests                  |
| docs/                   | This guide, the architecture document, and diagrams                      |

Each service folder also has its own README.md with service-specific run instructions and a description of its internal structure.

## 5. Try the End-to-End Flow Locally

With docker compose up running, trigger the two demo flows described in the architecture document:

```
# Low-risk refund – should auto-execute
curl -X POST localhost:8000/agents/refund/run \
  -H "Content-Type: application/json" \
  -d '{"agent_id": "demo-agent", "amount": 750}'

# High-risk refund – should return pending_approval
curl -X POST localhost:8000/agents/refund/run \
  -H "Content-Type: application/json" \
  -d '{"agent_id": "demo-agent", "amount": 25000}'
```

## 6. Coding Conventions

- Every new table must include `org_id` from the start, even in single-org mode — see the architecture document, section 6.
- The Gateway (Go) never calls the Control Plane's database directly, and vice versa — services communicate over HTTP only.
- Every governed action must produce an `audit_logs` entry — permission denials and policy decisions included, not just successful tool calls.
- Policy changes belong in `policies/*.rego`, mirrored to the `policies` table for the admin UI to display — the Gateway evaluates the Rego, not the database row.
- Mark unfinished integration points with `TODO(dev)` comments, as the current scaffold does — this makes it easy to grep for what's stubbed vs. real.

## 7. Branching & CI

- CI (`.github/workflows/ci.yml`) builds all four services independently on every push and pull request — a broken build in one service won't block review of unrelated changes.
- Open PRs against `main`; each service's CI job must pass before merge.

## 8. Deploying on Free Tiers (Current Bootstrap Phase)

The project is currently meant to run entirely on free hosting tiers — see the Bootstrap-to-Funded Stack document for the full reasoning. Quick reference:

| Component                   | Free Host                         | Notes                                                                                                                                                    |
|-----------------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend                    | Vercel or Netlify                 | Genuinely free, no card required, generous limits.                                                                                                       |
| Control Plane (Spring Boot) | Render free web service           | 512MB/0.1 CPU — Dockerfile already caps heap at -Xmx350m for this. Spins down after 15 min idle; expect a 30–60s cold start on first request after idle. |
| Gateway (Go)                | Render free web service           | Go's small footprint fits comfortably; same cold-start behavior applies.                                                                                 |
| Agent Runner                | Render free web service           | Only needed for demos; fine to run on the same free tier.                                                                                                |
| Database                    | Neon free Postgres                | 0.5GB storage, 100 compute-hours/month, scale-to-zero, no forced inactivity pause — more reliable for demos than Supabase's 7-day pause.                 |
| Cache                       | Upstash Redis free tier           | 500K commands/month, 256MB — plenty for early traffic.                                                                                                   |
| Policy Engine               | Self-hosted alongside the Gateway | OPA is open source — no separate hosting cost.                                                                                                           |

Watch memory on the Control Plane specifically — Spring Boot's JVM is the tightest fit on a 512MB free instance. If it becomes a problem, the architecture document explains why the Gateway (Go) and Control Plane (Spring Boot) are already split, which gives you room to move just the Control Plane to a larger free/cheap tier without touching the Gateway.

## 9. Where to Go Next

- Read docs/ARCHITECTURE.md (or the Word/PDF version) for the full request flow and design rationale before making structural changes.
- Read the Multi-Tenant SaaS Architecture document before adding anything that assumes multiple organizations.
- Check the TODO(dev) comments in services/gateway and services/control-plane — these mark the exact points where mocked logic needs to become real (permission lookups, OPA HTTP calls, tool integrations).